

需求預測與特徵選擇

10/07/2026 10:14:29

<https://chatgpt.com/c/6ac5a4b3-ca04-83e8-b43b-1fbce5c0ed52>

| **Store ID** | **Product ID** | **Category** | **Region** | **Inventory...**

可以。你這份資料很適合做「多元線性迴歸 + One-Hot Encoding + 特徵選擇 +...

我需要先做minmax 給我一段就好

可以，MinMax 只對數值欄位做，不要對 One-Hot 後的類別欄位做也可以。fro...

三、評分標準 ** 文件說明 (50%)** **** CRISP-DM 流程完整且邏...

照這個評分標準，你的作業重點其實很明確：不是只把模型跑出來，而是要把...



```

| **Store ID** | **Product ID** | **Category** |
**Region** | **Inventory Level** | **Units Sold** |
**Units Ordered** | **Price** | **Discount** | **Weather
Condition** | **Promotion** | **Competitor Pricing** |
**Seasonality** | **Epidemic** | **Demand** |   |
| -----: | -----: | -----: | -----: | -----
--: | -----: | -----: | -----: | -----
-----: | -----: | -----: | -----: | ----
-----: | -----: | --- |
|      0 |      S001 |      P0001 | Electronics |
North |      195 |      102 |      252 |      72.72 |
5 |      Snowy |      0 |      85.73 |
Winter |      0 | 115 |

```

資料如上 有好幾筆，關於類別，我想用onehot，但我不確定需不需要store id 和product id，如果需要 依樣用one hot，他的最終預測是demand

再來有以下需求:

2\ 分析任務\

使用線性回歸 (Linear Regression) 模型進行預測。\
 可嘗試單純線性回歸、多元線性回歸或 Auto Regression。\
 必須執行** 特徵選擇 (Feature
 Selection) **ȇ **模型評估 (Model
 Evaluation)**。\
 結果部分需包含請提供預測圖(加上信賴區間或預測區間)

3\ CRISP-DM 流程說明\

Business Understanding\
 Data Understanding\
 Data Preparation\
 Modeling\
 Evaluation\
 Deployment

2026/10/7 上午9:50:04



可以。你這份資料很適合做「**多元線性迴歸 + One-Hot Encoding + 特徵選擇 + Prediction Interval**」。

先講最重要的：

- Demand：目標變數 y
- Category / Region / Weather Condition / Promotion / Seasonality：類別型 → **One-Hot**
- Store ID / Product ID：先不要直接拿掉，但我建議做兩個模型比較
 - Model A：不使用 Store ID / Product ID
 - Model B：把 Store ID / Product ID 當類別 One-Hot
- Inventory Level / Units Sold / Units Ordered / Price / Discount / Competitor Pricing / Epidemic 等數值 → 保留
- 最後用特徵選擇決定哪些變數真正留下。

特別是 Store ID、Product ID 不能直接當 1、2、3、4 這種數字丟進 **Linear Regression**，因為那會讓模型誤以為 Store 4 比 Store 2 「大兩倍」。若要使用，就應該 One-Hot。

1. Store ID、Product ID 到底要不要？

我會建議你的作業做成：

先包含，再透過 Feature Selection 與模型比較判斷是否值得保留。

例如：

```
Store ID
S001
S002
S003
```

不能：

```
S001 → 1
S002 → 2
```

S003 → 3

而應：

Store_S001	Store_S002	Store_S003
1	0	0
0	1	0
0	0	1

但是 Store/Product 數量如果非常多，例如：

500 Stores
1000 Products

One-Hot 後會瞬間多出上千個變數，對普通線性迴歸不一定好。

所以你可以在報告中寫：

Store ID 與 Product ID 屬於識別型類別變數，本研究分別比較納入與排除 ID 特徵之線性迴歸模型，以模型評估結果判斷 ID 資訊是否有助於需求預測。

這樣反而很適合「Feature Selection」這項要求。

2. 你的欄位分類

假設真正資料長這樣：

Store ID
Product ID
Category
Region
Inventory Level
Units Sold
Units Ordered
Price
Discount
Weather Condition
Promotion
Competitor Pricing
Seasonality

Epidemic
Demand

我會分：

類型	欄位	處理
Target	Demand	不轉換
ID 類別	Store ID, Product ID	可 One-Hot，並做模型比較
一般類別	Category, Region, Weather Condition, Seasonality	One-Hot
Binary	Promotion, Epidemic	0/1 直接使用
Numeric	Inventory Level, Units Sold, Units Ordered, Price, Discount, Competitor Pricing	原值

不過你貼的表格看起來**可能多了一個 index 欄位造成資料錯位**。例如你現在顯示：

```
Store ID = 0
Product ID = S001
Category = P0001
Region = Electronics
Inventory Level = North
```

正常應該比較像：

```
index = 0
Store ID = S001
Product ID = P0001
Category = Electronics
Region = North
Inventory Level = 195
```

所以先跑：

```
print(df.columns)
print(df.head())
```

```
print(df.dtypes)
```

如果 CSV 最前面真的只是 index，就要先移掉。

3. 建議完整分析流程

你這份作業我會做：

```
graph TD
    A[原始資料] --> B[Data Understanding]
    B --> C[缺失值 / 重複值 / 型別確認]
    C --> D[One-Hot Encoding]
    D --> E[Train / Test Split]
    E --> F[Feature Selection]
    F --> G[Multiple Linear Regression]
    G --> H[MAE / RMSE / R²]
    H --> I[實際值 vs 預測值]
    I --> J[95% Prediction Interval]
```

Linear Regression 的概念就是利用多個特徵共同解釋 Demand；你這題不是單純只有一個 X，所以建議直接採 **Multiple Linear Regression**。

4. 第一部分：讀取資料

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv("your_data.csv")

print(df.head())
```

```
print(df.shape)
print(df.info())
```

如果有 `Unnamed: 0` :

```
df = df.loc[:, ~df.columns.str.contains("^Unnamed")]
```

檢查：

```
print(df.columns)
```

5. Data Understanding

缺失值

```
print(df.isnull().sum())
```

重複資料

```
print("Duplicated rows:", df.duplicated().sum())
```

數值統計

```
print(df.describe())
```

類別數量

你之前如果想寫 `groupby`，不用：

```
df[df.groupby(columns="Category").count()]
```

直接：

```
print(df["Category"].value_counts())
```

全部類別：

```
categorical_cols = [
    "Store ID",
    "Product ID",
    "Category",
    "Region",
    "Weather Condition",
    "Seasonality"
]

for col in categorical_cols:
    print(f"\n==== {col} =====")
    print(df[col].value_counts())
```

6. Data Preparation

先做一個「包含 Store ID / Product ID」版本。

```
target = "Demand"

categorical_cols = [
    "Store ID",
    "Product ID",
    "Category",
    "Region",
    "Weather Condition",
    "Seasonality"
]

X = df.drop(columns=[target])
y = df[target]
```

如果 `Promotion` 實際上是 Yes / No :

```
X["Promotion"] = X["Promotion"].map({
    "Yes": 1,
    "No": 0
})
```

如果本來就是 0/1 就不用。

7. One-Hot Encoding

最簡單：

```
X = pd.get_dummies(  
    X,  
    columns=categorical_cols,  
    drop_first=True,  
    dtype=int  
)
```

這裡：

```
drop_first=True
```

很重要。

因為 Linear Regression 要避免 dummy variable trap / 完全共線性。

例如 Region：

```
North  
South  
East  
West
```

只需要：

```
Region_South  
Region_East  
Region_West
```

North 自動成為 reference category。

檢查：

```
print(X.shape)  
print(X.head())
```

8. Train / Test Split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("Train:", X_train.shape)
print("Test:", X_test.shape)
```

9. Feature Selection

這是作業指定項目，所以一定要明確做出來。

我推薦你用：

SelectKBest + f_regression

因為你現在就是 Regression。

```
from sklearn.feature_selection import SelectKBest, f_regression

selector = SelectKBest(
    score_func=f_regression,
    k="all"
)

selector.fit(X_train, y_train)
```

取得每個 Feature 的分數：

```
feature_scores = pd.DataFrame({
    "Feature": X_train.columns,
    "F_Score": selector.scores_,
    "P_Value": selector.pvalues_
})

feature_scores = feature_scores.sort_values(
```

```
by="F_Score",
ascending=False
)

print(feature_scores)
```

例如結果可能：

Feature	F_Score
Units Sold	183.21
Price	91.34
Discount	73.55
Promotion	42.77
Seasonality_Winter	31.20
Weather_Condition_Snowy	18.32
...	

這張表就很適合直接放到報告。

10. 選擇顯著特徵

最簡單可以用：

```
P_Value < 0.05
```

```
selected_features = feature_scores[
    feature_scores["P_Value"] < 0.05
]["Feature"].tolist()

print("Selected features:")
print(selected_features)
```

然後：

```
X_train_selected = X_train[selected_features]
X_test_selected = X_test[selected_features]
```

如果因為資料特性導致一個都沒有：

```
if len(selected_features) == 0:
    selected_features = feature_scores.head(10)["Feature"].tolist()
```

11. Linear Regression

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()

model.fit(
    X_train_selected,
    y_train
)

y_pred = model.predict(X_test_selected)
```

12. Model Evaluation

建議至少放：

- MAE
- RMSE
- R^2

```
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

mae = mean_absolute_error(y_test, y_pred)

rmse = np.sqrt(
    mean_squared_error(y_test, y_pred)
)

r2 = r2_score(y_test, y_pred)
```

```
print("MAE :", mae)
print("RMSE:", rmse)
print("R² :", r2)
```

解釋：

MAE

→ 平均預測誤差多少 Demand

RMSE

→ 對大誤差懲罰較高，越低越好

R^2

→ 模型可以解釋多少 Demand 的變異

→ 越接近 1 越好

例如：

$R^2 = 0.82$

可以寫：

模型可解釋約 82% 的 Demand 變異。

13. 實際值 vs 預測值

這張建議一定畫。

```
plt.figure(figsize=(7, 6))

plt.scatter(
    y_test,
    y_pred,
    alpha=0.6
)

min_val = min(y_test.min(), y_pred.min())
max_val = max(y_test.max(), y_pred.max())
```

```
plt.plot(
    [min_val, max_val],
    [min_val, max_val],
    linestyle="--"
)

plt.xlabel("Actual Demand")
plt.ylabel("Predicted Demand")
plt.title("Actual vs Predicted Demand")

plt.grid(alpha=0.3)

plt.show()
```

虛線表示：

Actual = Prediction

點越靠近這條線越好。

14. 作業要求的「信賴區間 / 預測區間」

這裡我建議用 **statsmodels OLS** 。

因為 sklearn 的：

LinearRegression()

本身不方便直接給 Prediction Interval 。

使用 statsmodels

```
import statsmodels.api as sm

X_train_sm = sm.add_constant(
    X_train_selected.astype(float)
)

X_test_sm = sm.add_constant(
    X_test_selected.astype(float),
    has_constant="add"
```

```
)

ols_model = sm.OLS(
    y_train.astype(float),
    X_train_sm
).fit()

print(ols_model.summary())
```

這份 summary 也非常適合當作報告內容。

15. 取得 95% Prediction Interval

```
prediction = ols_model.get_prediction(X_test_sm)

prediction_summary = prediction.summary_frame(alpha=0.05)

print(prediction_summary.head())
```

會得到：

```
mean
mean_se
mean_ci_lower
mean_ci_upper
obs_ci_lower
obs_ci_upper
```

這裡要注意：

```
mean_ci_lower
mean_ci_upper
```

是 **Confidence Interval**。

而：

```
obs_ci_lower
obs_ci_upper
```

才是你預測單一 Demand 時更適合使用的：

Prediction Interval

所以建議作業畫 Prediction Interval。

16. 整理預測結果

```
results = pd.DataFrame({
    "Actual": y_test.values,
    "Predicted": prediction_summary["mean"].values,
    "Lower_PI": prediction_summary["obs_ci_lower"].values,
    "Upper_PI": prediction_summary["obs_ci_upper"].values
})

results = results.sort_values("Predicted").reset_index(drop=True)

print(results.head())
```

17. 畫 Prediction Interval

```
plt.figure(figsize=(12, 6))

x = np.arange(len(results))

plt.scatter(
    x,
    results["Actual"],
    label="Actual",
    alpha=0.7
)

plt.plot(
    x,
    results["Predicted"],
    label="Predicted"
)

plt.fill_between(
```

```
x,
results["Lower_PI"],
results["Upper_PI"],
alpha=0.2,
label="95% Prediction Interval"
)

plt.xlabel("Test Sample")
plt.ylabel("Demand")

plt.title(
    "Demand Prediction with 95% Prediction Interval"
)

plt.legend()

plt.grid(alpha=0.3)

plt.show()
```

這張就是最符合老師要求的：

預測圖 + 預測區間

18. 我建議再加一個「有沒有 ID」比較

這會讓你的 Feature Selection / Data Preparation 更有內容。

Model 1

包含 Store ID
包含 Product ID

Model 2

排除 Store ID
排除 Product ID

第二版：

```
X_no_id = df.drop(
    columns=[
        "Demand",
        "Store ID",
        "Product ID"
    ]
)
```

One-Hot：

```
categorical_cols_no_id = [
    "Category",
    "Region",
    "Weather Condition",
    "Seasonality"
]

X_no_id = pd.get_dummies(
    X_no_id,
    columns=categorical_cols_no_id,
    drop_first=True,
    dtype=int
)
```

最後比較：

Model	MAE	RMSE	R ²
With Store/Product ID	...	...	...
Without Store/Product ID	...	...	...

如果：

```
With ID R2 = 0.81
Without ID R2 = 0.80
```

那可以說：

Store ID 與 *Product ID* 僅帶來有限改善，因此考量模型簡潔性，最終模型排除識別型特徵。

反過來如果：

With ID = 0.91
Without ID = 0.72

就代表：

不同商店與商品確實具有明顯的需求差異，因此保留 *Store ID* 與 *Product ID* 的 One-Hot 特徵。

這比我們現在直接猜「要不要 ID」更合理。

19. CRISP-DM 可以怎麼寫

你的報告我建議直接按照六段寫。

1. Business Understanding

本研究目標為建立需求預測模型，根據商品銷售、價格、庫存、促銷、天氣、季節及區域等資訊預測商品 *Demand*。準確的需求預測可協助企業進行庫存管理、補貨決策及促銷規劃，降低缺貨或庫存過剩的風險。

2. Data Understanding

資料包含 *Store ID*、*Product ID*、*Category*、*Region*、*Inventory Level*、*Units Sold*、*Units Ordered*、*Price*、*Discount*、*Weather Condition*、*Promotion*、*Competitor Pricing*、*Seasonality*、*Epidemic* 與 *Demand* 等欄位。首先進行資料型態、缺失值、重複值及描述性統計分析，並觀察不同類別及數值特徵的分布情形。

3. Data Preparation

將 *Demand* 設為目標變數，其餘欄位作為候選特徵。*Category*、*Region*、*Weather Condition*、*Seasonality* 等類別型變數採 One-Hot Encoding 轉換。*Store ID* 與 *Product ID*

同樣視為類別變數，並另外比較納入及排除 ID 特徵的模型表現。資料之後切分為 80% 訓練集與 20% 測試集。

4. Modeling

本研究採用 *Multiple Linear Regression* 預測 Demand。模型建立前使用 *F-test* (*f_regression*) 進行特徵選擇，評估各候選變數與 Demand 的關聯程度，並選擇較具解釋力的特徵建立最終迴歸模型。

5. Evaluation

使用 MAE、RMSE 與 R^2 評估模型表現。此外，比較實際 Demand 與模型預測值，並利用 OLS 模型建立 95% *Prediction Interval*，以呈現模型預測的不確定性。同時比較納入與排除 Store ID / Product ID 時的模型效能。

6. Deployment

完成的需求預測模型可整合至庫存與補貨系統。當系統取得商品價格、促銷、庫存、天氣及季節等資訊後，即可輸出預測 Demand 及其預測區間，協助管理者進行補貨、庫存配置與促銷決策。

我最推薦你最後的作業架構

你不用搞 Auto Regression，因為從你目前貼的資料看，**沒有明確的時間欄位**。因此最合理的是：

Multiple Linear Regression

↓

One-Hot Encoding

Category

Region

Weather

Seasonality

Store ID / Product ID

↓

Feature Selection

F-test / p-value

↓

Linear Regression

↓

MAE
RMSE
 R^2

↓

Actual vs Predicted

↓

95% Prediction Interval

其中 `Store ID / Product ID` 我建議不要現在就武斷刪除，而是做「有 ID / 無 ID」兩個模型，用測試集的 **RMSE 與 R^2** 決定。這樣你在報告裡同時有「特徵選擇」和「實驗比較」，完整度會高很多。

2026/10/7 上午9:50:09



我需要先做minmax 給我一段就好

2026/10/7 上午10:00:07



可以，**MinMax** 只對數值欄位做，不要對 **One-Hot** 後的類別欄位做也可以。

```
from sklearn.preprocessing import MinMaxScaler

numeric_cols = [
    "Inventory Level",
    "Units Sold",
    "Units Ordered",
    "Price",
    "Discount",
    "Competitor Pricing"
]

scaler = MinMaxScaler()

df[numeric_cols] = scaler.fit_transform(df[numeric_cols])

df.head()
```

這樣會把這些數值欄位縮放到 **0~1**。

Demand 建議先不要 MinMax，因為它是最後要預測的目標值。

2026/10/7 上午10:00:15



三、評分標準

** 文件說明（50%）**\

CRISP-DM 流程完整且邏輯清楚（25%）\

包含 GPT 對話與 NotebookLM 摘要（15%）\

有明確說明資料集來源與研究脈絡（10%）

\
** 結果呈現（50%）**\

模型正確可執行，具特徵選擇與評估（25%）\

結果合理、美觀且具有說服力（15%）\

呈現出Kaggle名次(若有)/預測結果評估(預測圖、評估指標)
(10%)

2026/10/7 上午10:05:04



照這個評分標準，你的作業重點其實很明確：**不是只把模型跑出來，而是要把 CRISP-DM、特徵選擇、模型評估、預測圖完整串起來。**

我建議你最後 Notebook / 報告就照下面順序做，會最對應評分。

1. 資料集來源與研究脈絡

- 資料集從哪裡來，例如 Kaggle
- 資料共有幾筆、幾個欄位
- 研究問題：利用庫存、價格、促銷、天氣、季節等資訊預測 Demand
- 為什麼需求預測重要：補貨、庫存管理、降低缺貨與過量庫存

2. CRISP-DM 六階段

- Business Understanding：預測商品需求
- Data Understanding：欄位、資料型態、缺失值、描述統計
- Data Preparation：缺失處理、MinMax、One-Hot、Train/Test Split
- Modeling：Linear Regression
- Evaluation：MAE、RMSE、 R^2 、Prediction Interval
- Deployment：未來可用於庫存與補貨決策

3. 模型部分一定要明確做 Feature Selection

你前面做的：

```
from sklearn.feature_selection import SelectKBest, f_regression

selector = SelectKBest(score_func=f_regression, k="all")
selector.fit(X_train, y_train)

feature_scores = pd.DataFrame({
    "Feature": X_train.columns,
    "F_Score": selector.scores_,
    "P_Value": selector.pvalues_
})

feature_scores = feature_scores.sort_values(
    "F_Score",
    ascending=False
)

feature_scores
```

接著：

```
selected_features = feature_scores[
    feature_scores["P_Value"] < 0.05
]["Feature"].tolist()

selected_features
```

這樣老師就很容易看到你**真的有做特徵選擇**，而不是只在文字寫有做。

4. 結果至少放這 4 個

- MAE
- RMSE
- R^2
- 預測圖 + 95% Prediction Interval

另外我會再加一張：

Actual Demand vs Predicted Demand

這樣結果呈現會更完整。

你的結果章節大概可以長這樣：

Model Evaluation

MAE = 8.42
RMSE = 11.37
 R^2 = 0.84

再搭配：

圖 1：Actual vs Predicted Demand

圖 2：Demand Prediction with 95% Prediction Interval

這基本上就直接對應「預測結果評估（預測圖、評估指標）」那 10%。

至於 **GPT 對話與 NotebookLM 摘要 15%**，你也要特別留一小節，不要忘記。可以放：

本研究使用 *ChatGPT* 協助討論資料前處理、類別變數編碼、*MinMax Scaling*、特徵選擇方法及模型評估方式；另使用 *NotebookLM* 對相關資料與分析內容進行摘要與整理。

然後附上幾張對話截圖、NotebookLM 摘要截圖即可。

如果你想把分數拿穩，我會建議你的 Notebook 標題直接排成：

1. Business Understanding
2. Data Understanding
3. Data Preparation
4. Feature Selection
5. Modeling
6. Evaluation
7. Deployment
8. GPT Conversation
9. NotebookLM Summary
10. Conclusion

這樣老師光看目錄就能直接對上評分表，不容易漏分。

2026/10/7 上午10:05:07